

Irrigation Scheduling Design

04/20/2026 18:20:29

I am designing the logic of my irrigation system in python. I am hesitating between...

Short answer: go with B (smarter scheduling) — but implement it with a clear...

I listed option A and B, could it be I missed an option C that could fit me?

Yes — and this is exactly the kind of system where a strong Option C can outperform...

Give me the best type of scheduling that I could do, tell me if it is even another...

If you want the best long-term design, it's not A, B, or even just C as described...

What could be next step to make this even better, like a really smart scheduling

If you're already thinking beyond A/B/C/D, you're basically entering the territory of...

Yes give me that and also answer next 3 questions . Lawn Vs trees behave...

Great — now you're asking the right questions. This is exactly where irrigation...

build a full example (3 zones: lawn, tree, shrubs) with real scheduling + adaptation

Alright — let's build a complete, realistic mini-system that ties everything together: ...

Would it be interesting to make the scheduling respect exactly the soaking time ?

Short answer: yes — but not strictly. The best system respects soaking time as a...

How would I make it so it automatically schedule each day, I am running fast API f...

Nice — now you're moving from "algorithm" to an actual running system. You...



I am designing the logic of my irrigation system in python.

I am hesitating between:

A. basic scheduling (setting for each zone a time during the day and duration, with option of choosing every X day and cycle and soak, meaning I have to manually put each slot one after the other, or make sure they don't clash)

B. a bit smarter scheduling (choosing for each zone if it is gonna run the morning/evening and every X day and cycle and soak, but the program would optimise the watering by putting each slot after the other, even when soaking)

29/03/2026, 01:26:54



Short answer: go with **B (smarter scheduling)** — but implement it with a *clear separation between "what the user wants" and "how it's scheduled."*

Let's break this down practically from a system design perspective.

Why A (basic scheduling) will hurt you later

At first glance, A feels simpler because it's very explicit. But in reality:

- ✗ Users must manually avoid overlaps → error-prone
- ✗ Hard to adjust when adding/removing zones
- ✗ Cycle & soak becomes painful (lots of micro-slots)
- ✗ Doesn't scale (10+ zones becomes a nightmare)
- ✗ Logic gets duplicated in the UI instead of code

You're basically pushing scheduling complexity onto the user.

Why B (smarter scheduling) is the better foundation

B is much closer to how real irrigation controllers work.

- ✓ User expresses **intent**, not exact timing
- ✓ Your program ensures **no conflicts automatically**
- ✓ Easier to add features later (weather, priorities, etc.)

-  Cleaner mental model: "what" vs "when"
-  Cycle & soak becomes natural to handle programmatically

The key design insight

Think in two layers:

1. User Intent Layer (configuration)

Example:

```
ZoneConfig(  
    name="Front Lawn",  
    frequency_days=2,  
    preferred_windows=["morning"],  
    duration=20,  
    cycle_soak=(5, 10) # 5 min water, 10 min soak  
)
```

2. Scheduling Engine (your logic)

This converts intent → actual timeline:

```
06:00 Zone A (5 min)  
06:05 Zone B (5 min)  
06:10 Zone C (5 min)  
06:15 Zone A (5 min) # after soak  
...
```

👉 This is where the real value is.

The tricky part: Cycle & Soak with optimization

This is exactly why B shines.

Instead of:

Zone A: 5min → wait 10min → 5min

You can **interleave zones**:

A(5) → B(5) → C(5) → A(5) → B(5) → C(5)

So:

- Soak time is *naturally respected*
- Total watering time is minimized
- No idle gaps

Suggested scheduling algorithm (simple & solid)

1. Filter zones that should run **today**
2. Split each zone into **cycles**
3. Create a queue of “watering tasks”
4. Loop through tasks in round-robin
5. Respect:
 - soak delay (don’t reuse a zone too early)
 - time windows (morning/evening)

Rough pseudo-code

```
schedule = []
time = start_of_window

while tasks_remaining:
    for zone in zones:
        if zone.can_run_now(time):
            schedule.append((time, zone, zone.cycle_duration))
```

```
zone.mark_run(time)
time += zone.cycle_duration
```

Important design decisions you should make now

1. Time windows

Define them clearly:

```
MORNING = (5:00, 10:00)
EVENING = (18:00, 22:00)
```

2. Conflict resolution strategy

What if you run out of time in a window?

Options:

- Cut remaining cycles ❌ (bad for plants)
- Spill into next window ✅ (better)
- Shift start time earlier/later ✅ (smartest)

3. Zone priority (future-proofing)

Even if you don't use it yet:

```
priority=1 # trees > grass
```

4. Deterministic vs adaptive

Start with deterministic (same input → same schedule).

Later you can extend to:

- weather-based skipping

- soil moisture feedback
-

Hybrid approach (best of both worlds)

You can still allow power users to override:

- Default → smart scheduling (B)
 - Optional → “fixed start time” per zone (A-style override)
-

Bottom line

- A = simple code, painful UX
- B = slightly harder code, much better system

👉 In real systems, **scheduling logic is the product** — not the UI.

If you want, I can help you:

29/03/2026, 01:26:54

- design the actual Python classes
- implement the scheduling engine
- or simulate a real schedule with sample zones



I listed option A and B, could it be I missed an option C that could fit me?

29/03/2026, 01:30:03



Yes — and this is exactly the kind of system where a strong **Option C** can outperform both A and B.

What you're missing is a **constraint-based / goal-driven scheduler**.

Option C: Constraint-Based Scheduling (Hybrid Smart Engine)

Instead of:

- A → "user defines exact times"
- B → "system arranges sequentially"

👉 C = **user defines rules + constraints, system solves a schedule**

How Option C works

The user doesn't say *when exactly* or even strictly *morning/evening*.

They define things like:

```
ZoneConfig(  
  name="Front Lawn",  
  water_total=20,           # total minutes needed  
  max_continuous=5,         # for cycle & soak  
  soak_time=10,  
  frequency_days=2,  
  allowed_windows=["morning", "evening"],  
  deadline="10:00",         # must finish before this  
  priority=2  
)
```

Then your engine:

👉 **builds the best schedule that satisfies all constraints**

🧩 Why this is different from B

B is still fairly rigid:

- fixed “morning OR evening”
- simple sequential placement

C introduces:

- ✅ flexibility inside windows
 - ✅ deadlines instead of fixed starts
 - ✅ optimization (minimize runtime, gaps, etc.)
 - ✅ prioritization
-

💡 Real-world analogy

Think:

- A = Google Calendar manual events
 - B = “auto arrange my meetings back-to-back”
 - C = **Google Calendar + “find a time” + constraints + priorities**
-

🚀 What Option C unlocks

1. True optimization

Example goal:

- finish watering **as early as possible**
 - or **minimize total runtime**
 - or **avoid peak evaporation hours**
-

2. Smarter cycle & soak

Instead of hardcoding behavior:

- “wait 10 min” becomes a **constraint**
 - scheduler naturally fills gaps with other zones
-

3. Conflict resolution becomes automatic

Instead of:

“these overlap, what do I do?”

The system:

👉 rearranges everything to fit constraints

4. Future features become easy

You can later plug in:

- 🌧️ weather forecast → reduce watering
- 🌱 soil type → change soak times
- 💧 water pressure limits → restrict concurrent zones
- ⚡ energy tariffs → avoid peak hours

Without redesigning everything.

🧠 Implementation approach (keep it simple)

You don’t need a full-blown solver.

Start with a **greedy constraint scheduler**:

1. Expand zones into tasks (cycles)
 2. Maintain a timeline
 3. At each step, pick the “best valid task”:
 - allowed at current time
 - soak constraint satisfied
 - highest priority
 - closest to deadline
-

Pseudo-logic

```
while tasks_remaining:
    candidates = [
        task for task in tasks
        if task.is_allowed(now)
    ]

    best = select_best(candidates)

    schedule.append(best)
    best.mark_done(now)

    now += best.duration
```

Trade-offs

Option	Complexity	Flexibility	UX	Scalability
A	★	✗	✗	✗
B	★★★	✓	✓	⚠
C	★★★★	🚀	✓ (if designed well)	🚀

My recommendation

Don't choose strictly A, B, or C.

👉 Build **B** with a **C-style** core

Meaning:

- UI looks like B (simple: morning/evening, every X days)
- Engine behaves like C (constraint solver underneath)

Key insight

If you design around **time slots**, you'll get stuck.

If you design around **constraints + tasks**, you'll scale.

If you want, I can:

- help you design the data model for Option C
- or write a minimal working scheduler in Python
- or stress-test your logic with real-world scenarios (many zones, tight windows, etc.)

29/03/2020, 01:50:02



Give me the best type of scheduling that I could do, tell me if it is even another option

29/03/2026, 01:32:47



If you want the **best long-term design**, it's not A, B, or even just C as described earlier.

👉 The strongest approach is:

🧠 Option D: Intent-Based + Constraint Solver + Simulation (Hybrid Adaptive Scheduler)

This is how you'd design it if you wanted something **robust, scalable, and "pro-grade"**, while still usable.

💡 The idea in one sentence

Users describe **watering goals**, your system:

1. **translates them into tasks**
2. **builds a schedule using constraints**
3. **simulates & adjusts it before execution**

🔍 Why this is different (and better)

- A = manual timeline
- B = auto ترتيب (but rigid logic)
- C = constraint-based

👉 **D adds a missing layer: simulation + validation + adaptation**

This is what makes it *reliable in real life*, not just "smart on paper."

🏗️ The 3-layer architecture

1. 🌱 Intent Layer (what the user wants)

User defines goals, not schedules:

```
ZoneIntent(  
    name="Front Lawn",  
    water_minutes=20,  
    frequency_days=2,  
    preferred_windows=["morning"],  
    soak_time=10,  
    max_cycle=5,  
)
```

2. Planning Layer (constraint-based scheduler)

This is your engine:

- splits into cycles
- respects soak delays
- fits inside time windows
- avoids overlaps
- orders zones intelligently

3. Simulation Layer (this is the “Option D upgrade”)

Before finalizing:

- simulate the full day
- detect:
 -  unfinished zones
 -  window overflow
 -  impossible constraints
- auto-fix or warn

Example

User says:

- 6 zones
- all in morning
- each needs 30 min total

Simulation detects:

👉 impossible before 10:00

System responds:

- shift some zones to evening ✅
- or start earlier ✅
- or notify user !

🚀 What makes this the *best* approach

✅ 1. Handles real-world complexity

- limited time windows
- many zones
- cycle & soak interleaving
- future constraints (weather, pressure)

✅ 2. Prevents silent failures

A/B/C can produce “valid-looking but bad” schedules.

D:

👉 verifies before execution

✅ 3. Extensible without redesign

You can later add:

- ☁️ rain skipping
- 🌡️ evapotranspiration adjustment
- 💧 flow limits (no 2 zones at once)
- ⚡️ smart energy timing

Without breaking your model.

✅ 4. UX stays simple

User still sees something like:

- Morning / Evening
- Every X days
- Duration

👉 but under the hood, it's much smarter

⚙️ Implementation strategy (keep it practical)

You do NOT need a heavy solver.

Start with:

Step 1: Expand intents into tasks

Zone → cycles → tasks

Step 2: Greedy + constraint scheduling

- only pick valid tasks
 - respect soak delays
 - stay inside windows
-

Step 3: Run a simulation pass

```
if not all_tasks_scheduled:  
    adjust_plan()
```

Step 4: Optional optimization pass

- reduce idle time
 - reorder zones
 - compress schedule
-

Key design principle

👉 Never store schedules as fixed times

Always store:

- intents
- constraints

And regenerate schedules dynamically.

⚠️ What to avoid

- Hardcoding time slots (A trap)
 - Overfitting to morning/evening only (B limitation)
 - Building a full SAT/linear solver too early (overkill)
-

🏁 Final recommendation

👉 Build **Option D-lite**:

- Intent-based config (like B)
 - Constraint scheduler (like C)
 - Simple simulation check (the D upgrade)
-

🔥 Mental model

Think of your system as:

"Given constraints, generate the best possible watering plan for today."

Not:

"Store and execute predefined times."

If you want, next step I can:

- design your Python class architecture cleanly
- or write a working scheduling engine you can plug in immediately

29/03/2026, 01:32:47



What could be next step to make this even better, like a really smart scheduling

29/03/2026, 01:34:30



If you're already thinking beyond A/B/C/D, you're basically entering the territory of **adaptive, autonomous systems** — the kind used in high-end irrigation controllers and agriculture.

So the real "next step" isn't just better scheduling...

👉 it's **making scheduling self-adjusting based on reality**.

Option E: Adaptive, Feedback-Driven Scheduling (Self-Tuning System)

This is the "really smart" version.

Instead of:

"generate the best schedule"

👉 the system becomes:

"continuously learn and adjust watering over time"



Core idea

Your system becomes a loop:

Plan → Execute → Measure → Adjust → Repeat



What you add on top of Option D

1. External signals (environment awareness)

You integrate real-world inputs:



- Weather forecast (rain, temperature, wind)
- Evapotranspiration (ET)
- Seasonality

👉 Example:

- skip watering if rain is coming
- increase duration during heatwaves

2. 🌱 Soil / outcome feedback (this is the big leap)

If you can (even later):

- soil moisture sensors
- or inferred dryness (no sensor)

Then:

👉 stop guessing → react to actual conditions

3. 📈 Learning layer (even simple heuristics work)

You don't need AI to start.

Example rules:

```
if soil_too_wet:
    reduce_duration(zone)

if soil_too_dry:
    increase_frequency(zone)
```

Over time:

👉 each zone becomes tuned to its real needs

4. 🎯 Goal-based watering (instead of fixed duration)

Instead of:

```
water_minutes = 20
```

You move to:

```
target_moisture = 0.25
```

System decides:

👉 how long to water to reach that

What this unlocks

✅ True efficiency

- no overwatering
- no underwatering

✅ Per-zone intelligence

- lawn vs trees behave differently automatically

✅ Climate adaptation

- summer ≠ spring ≠ autumn

✅ Less user input over time

System gets better → user does less

Practical roadmap (important)

Don't jump straight to full "AI irrigation".

Build in stages:

◆ Stage 1 (what you're doing now)

✓ Intent-based + constraint scheduler (Option D)

◆ Stage 2 (easy win)

Add **weather awareness**

- skip if rain > X mm
 - scale duration with temperature
-

◆ Stage 3

Add **dynamic coefficients per zone**

```
zone_factor = 1.2 # dry area
```

◆ Stage 4

Add **feedback loop (even without sensors)**

- track:
 - how often watering happens
 - seasonal adjustments
-

◆ Stage 5 (advanced)

Add **real sensors or predictive model**

⚠ Reality check

You don't need machine learning to be "smart".

Most real systems are:

👉 80% rules + 20% adaptation

And they work extremely well.

🔥 The real “next-level” insight

The smartest system is not the one that:

schedules perfectly once

It's the one that:

keeps correcting itself forever

🏁 Final answer

👉 The best possible evolution is:

Option E = Adaptive, feedback-driven irrigation system

Built on top of:

- Intent-based model (D)
 - Constraint scheduling
 - Simulation
 - - real-world feedback loop
-

If you want, I can next:

design a clean Python architecture for Option D → E transition

- or show how to plug in weather APIs and make decisions
- or simulate a full adaptive watering cycle over a week



Yes give me that and also answer next 3 questions .

Lawn Vs trees behave differently, does that mean that one would next constant moisture level and the other sporadic watering ?

What would be needed from the user to have implement Evapotranspiration?

Also tell me how I could get inferred dryness?

29/03/2026, 01:40:07



Great — now you're asking the *right* questions. This is exactly where irrigation systems go from "scheduler" to "smart system."

I'll do this in 4 parts:

1. Clean Python architecture (D → E ready)
2. Lawn vs trees (very important concept)
3. What you need for evapotranspiration (ET)
4. How to infer dryness without sensors



1. Python Architecture (designed for evolution)

The key is **separating concerns** so you can plug intelligence later without rewriting everything.

Core modules

```
irrigation/  
├─ models.py          # data structures  
├─ planner.py         # builds schedule (Option D)  
├─ simulator.py       # validates schedule  
└─ executor.py        # runs schedule
```

```
|— environment.py    # weather, ET, external data
|— adaptation.py     # learning / adjustments (Option E)
```

models.py (stable foundation)

```
from dataclasses import dataclass

@dataclass
class Zone:
    name: str
    plant_type: str # "lawn", "tree", "shrub"
    soil_type: str  # "sand", "loam", "clay"
    sun_exposure: str # "full", "partial", "shade"

@dataclass
class ZoneIntent:
    zone: Zone
    base_minutes: int
    frequency_days: int
    preferred_windows: list[str]
    max_cycle: int
    soak_time: int
```

planner.py (your current focus)

- Converts intent → tasks → schedule
- No intelligence here, just rules

environment.py (future power)

```
@dataclass
class Environment:
    temperature: float
```



```
rain_forecast_mm: float
evapotranspiration: float # mm/day
```

adaptation.py (this is the upgrade layer)

```
def adjust_duration(intent, env, dryness_index):
    factor = 1.0

    # ET adjustment
    factor *= env.evapotranspiration / 4.0 # baseline

    # dryness adjustment
    factor *= dryness_index

    return intent.base_minutes * factor
```

👉 Notice:

- planner doesn't care
- adaptation modifies inputs

2. Lawn vs Trees (critical concept)

Yes — and this is *fundamental*.

Lawn (grass)

- shallow roots (~5–15 cm)
- dries quickly
- needs **frequent, light watering**

👉 Goal:

*maintain relatively **stable moisture***

Example:

- every 1–2 days
- shorter duration

- cycle & soak important



Trees / shrubs

- deep roots (30 cm → several meters)
- drought resistant
- prefer **deep, infrequent watering**

👉 Goal:

soak deeply, then let soil dry

Example:

- every 5–10 days
- long watering
- minimal cycling



Key difference

Type	Strategy
Lawn	Keep topsoil moist
Trees	Encourage deep root growth



What this means for your system

You should NOT treat all zones equally.

Add a layer:

```
def default_strategy(zone):  
    if zone.plant_type == "lawn":  
        return {"freq": 2, "cycle": 5}  
    elif zone.plant_type == "tree":  
        return {"freq": 7, "cycle": 20}
```



3. What you need for Evapotranspiration (ET)

You have 3 levels of complexity:

✅ Level 1 (simple & practical — recommended)

You only need:

- temperature
- optionally: humidity / wind (bonus)

Then use a simplified model:

```
def estimate_et(temp):  
    return 0.1 * temp # rough mm/day
```

👉 crude but useful

✅ Level 2 (best balance)

Use a weather API that gives ET directly.

User needs:

- location (lat/lon)
- nothing else

You fetch:

- ET_0 (reference evapotranspiration)
-

Level 3 (full model, probably overkill)

Full **Penman-Monteith equation** needs:

- temperature
- humidity
- solar radiation
- wind speed

👉 not worth implementing yourself unless you want a science project

💡 How you use ET

ET tells you:

how much water the soil loses per day

So:

```
adjustment_factor = et_today / et_baseline
```

Example:

- ET = 6 mm → hot → increase watering
- ET = 2 mm → cool → reduce

🌵 4. Inferred dryness (no sensors)

This is the fun part — and very doable.

You simulate a **virtual soil moisture model**.

🧠 Core idea

Track a value per zone:

```
soil_moisture = 0.0 → 1.0
```

🔄 Update over time

Each day:

```
soil_moisture -= evapotranspiration_loss  
soil_moisture += rain  
soil_moisture += irrigation
```

💧 Example

```
def update_moisture(m, et, rain, irrigation):  
    m -= et * 0.05  
    m += rain * 0.1  
    m += irrigation * 0.1  
    return max(0, min(1, m))
```

Then use it

```
if soil_moisture < 0.3:  
    increase_watering()  
  
if soil_moisture > 0.8:  
    skip_watering()
```

Why this works surprisingly well

Even without sensors:

- ET = drying force
- rain = natural input
- irrigation = controlled input

👉 you get a **good approximation of reality**

Make it zone-specific

Different soils behave differently:

```
SOIL_RETENTION = {  
    "sand": 0.3,  
    "loam": 0.6,  
    "clay": 0.8  
}
```

👉 sand dries fast, clay holds water

🏁 Final synthesis

You're building:

Today (D)

- intent-based scheduling
- constraint solver
- simulation

Tomorrow (E)

- ET-driven adjustments
 - plant-type strategies
 - inferred soil moisture
 - adaptive feedback loop
-

🔥 The real "pro move"

Don't try to be perfect.

👉 Just start tracking:

- ET
- watering
- estimated moisture

And adjust slightly over time.

That alone puts you ahead of most commercial systems.

29/03/2026, 01:40:06

If you want next step, I can:

- write a working Python "soil moisture simulator"
- or build a full example (3 zones: lawn, tree, shrubs) with real scheduling + adaptation



build a full example (3 zones: lawn, tree, shrubs) with real scheduling + adaptation

29/03/2026, 01:50:23



Alright — let's build a **complete, realistic mini-system** that ties everything together:

- 3 zones (lawn, tree, shrubs)
- intent-based config
- scheduling (cycle & soak, sequential)
- ET + rain adaptation
- inferred soil moisture

This will be **simple but structurally correct**, so you can extend it.

Full Example: Smart Irrigation System (D → E)

1. Models

```
from dataclasses import dataclass

@dataclass
class Zone:
    name: str
    plant_type: str    # "lawn", "tree", "shrub"
    soil_type: str     # "sand", "loam", "clay"

@dataclass
class ZoneState:
    moisture: float    # 0.0 → 1.0

@dataclass
class ZoneIntent:
    base_minutes: int
    frequency_days: int
```

```
max_cycle: int
soak_time: int
```

2. 🌍 Environment (ET + rain)

```
@dataclass
class Environment:
    temperature: float
    rain_mm: float

def estimate_et(temp):
    return 0.1 * temp # simple ET model (mm/day)
```

3. 🌵 Soil model (inferred dryness)

```
SOIL_RETENTION = {
    "sand": 0.3,
    "loam": 0.6,
    "clay": 0.8
}

def update_moisture(state, zone, env, irrigation_mm):
    et = estimate_et(env.temperature)

    # drying
    state.moisture -= et * (1 - SOIL_RETENTION[zone.soil_type]) * 0.05

    # rain input
    state.moisture += env.rain_mm * 0.02

    # irrigation input
    state.moisture += irrigation_mm * 0.05

    return max(0, min(1, state.moisture))
```


4. 🧠 Adaptation logic

```
def adjust_duration(intent, zone, state, env):
    et = estimate_et(env.temperature)

    factor = 1.0

    # ET adjustment
    factor *= et / 4.0 # baseline

    # dryness adjustment
    if state.moisture < 0.3:
        factor *= 1.3
    elif state.moisture > 0.7:
        factor *= 0.7

    # plant type adjustment
    if zone.plant_type == "tree":
        factor *= 1.2
    elif zone.plant_type == "lawn":
        factor *= 0.9

    return max(5, int(intent.base_minutes * factor))
```

5. ⚙️ Scheduler (cycle & soak interleaving)

```
from collections import deque

def build_schedule(zones, intents, durations):
    tasks = []

    for zone in zones:
        total = durations[zone.name]
        cycle = intents[zone.name].max_cycle

        cycles = []
        while total > 0:
```

```

        run = min(cycle, total)
        cycles.append(run)
        total -= run

    tasks.append({
        "zone": zone,
        "cycles": deque(cycles),
        "soak": intents[zone.name].soak_time,
        "next_available": 0
    })

schedule = []
time = 0

while any(t["cycles"] for t in tasks):
    for t in tasks:
        if t["cycles"] and time >= t["next_available"]:
            duration = t["cycles"].popleft()

            schedule.append((time, t["zone"].name, duration))

            time += duration
            t["next_available"] = time + t["soak"]

return schedule

```

6. 🌿 Define your 3 zones

```

zones = [
    Zone("Lawn", "lawn", "loam"),
    Zone("Tree", "tree", "clay"),
    Zone("Shrubs", "shrub", "sand")
]

states = {
    "Lawn": ZoneState(0.5),
    "Tree": ZoneState(0.6),
    "Shrubs": ZoneState(0.4)
}

```

```
}

intents = {
    "Lawn": ZoneIntent(20, 2, 5, 10),
    "Tree": ZoneIntent(40, 7, 20, 30),
    "Shrubs": ZoneIntent(25, 3, 8, 15)
}
```

7. 🌤️ Simulate a day

```
env = Environment(
    temperature=28,    # warm day
    rain_mm=0
)
```

8. 🧠 Compute adapted durations

```
durations = {}

for zone in zones:
    durations[zone.name] = adjust_duration(
        intents[zone.name],
        zone,
        states[zone.name],
        env
    )
```

9. ⚙️ Build schedule

```
schedule = build_schedule(zones, intents, durations)
```

```
for t in schedule:
    print(f"{t[0]:03d} min → {t[1]} for {t[2]} min")
```

10. 💧 Apply watering + update moisture

```
for _, zone_name, duration in schedule:
    irrigation_mm = duration * 0.2 # rough conversion

    zone = next(z for z in zones if z.name == zone_name)

    states[zone_name].moisture = update_moisture(
        states[zone_name],
        zone,
        env,
        irrigation_mm
    )
```

🔍 What this system actually does

Lawn

- shorter cycles (5 min)
- frequent
- moisture kept stable

Tree

- long cycles (20 min)
- deep watering
- long soak

Shrubs

- medium behavior

🧠 What makes this “smart”

- ET increases watering on hot days
- moisture reduces watering if already wet
- soil type changes drying speed
- cycle & soak interleaved automatically
- no overlaps ever

Example outcome (conceptually)

```
000 → Lawn 5 min
005 → Tree 20 min
025 → Shrubs 8 min
033 → Lawn 5 min
...
```

👉 Efficient, no idle time, soak respected

Where to go next

If you want to push this further, the next upgrades are:

1. Time windows (morning/evening constraint)

2. Rain skip logic

```
if env.rain_mm > 5:
    skip_watering()
```

3. Multi-day simulation loop

4. Persist moisture state over days

Final takeaway

You now have a system that:

- **plans** (scheduler)
- **adapts** (ET + moisture)
- **simulates reality** (soil model)

👉 That's already close to commercial smart irrigation controllers.

If you want, next I can:

- extend this to a **7-day simulation with weather changes**
- or add **morning/evening constraints + deadline-based scheduling**
- or turn this into a **clean Python package structure with tests**



Would it be interesting to make the scheduling respect exactly the soaking time ?

29/03/2026, 03:09:23



Short answer: **yes — but not strictly.**

The *best system respects soaking time as a minimum constraint, not an exact one.*

Let me explain why this matters in practice.



The key distinction

There are two ways to interpret soak time:

✗ Strict (exact timing)

"Run next cycle exactly 10 minutes later"



✓ Flexible (minimum constraint)

"Do not run again before 10 minutes — but later is fine"



What happens in real soil

Soak time is not a precise timer like cooking pasta.

It represents:

- water infiltration
- redistribution in soil
- avoiding runoff

👉 These processes are **variable**, not exact:

- soil type changes it (sand vs clay)
- slope matters
- temperature matters



⚠️ Why strict soak timing is a bad idea

If you enforce exact timing:

1. You create artificial inefficiency

Zone A: `wait` EXACTLY `10` min
(`no` other zone fits perfectly)
→ `system` idle ❌

2. You over-constrain your scheduler

It becomes:

- harder to compute
- more fragile
- less adaptable

3. You gain nothing agronomically

Plants don't care if it's:

- 10 min vs 14 min soak

👉 but they do care about:

- **minimum soak respected**

✅ The correct model

Treat soak time as:

`next_available_time` = `last_run` + `soak_time`

👉 and that's it.

Then your scheduler is free to:

- insert other zones
- delay slightly if needed
- optimize globally

This unlocks a powerful behavior

Your system naturally does:

```
A → B → C → A → B → C
```

Instead of:

```
A → wait → A → wait → A ❌
```

👉 This is **exactly what professional controllers aim for**

Advanced refinement (if you want to be really smart)

You can even make soak time *adaptive*:

```
def effective_soak(zone, env):  
    base = zone.soak_time  
  
    if zone.soil_type == "clay":  
        base *= 1.5 # slower infiltration  
  
    if env.temperature > 30:  
        base *= 0.8 # dries faster  
  
    return base
```

When would exact soak timing ever make sense?

Almost never, but edge cases:

- very steep slopes (erosion risk)
- extremely precise irrigation systems
- experimental setups

👉 even then, it's still usually a **minimum**, not exact

🏁 Final recommendation

👉 Implement soak time as:

- ✅ **minimum delay constraint**
 - ❌ NOT exact scheduling target
-

🔥 The deeper insight

You're not scheduling *events in time*.

You're scheduling under **constraints**:

- "not before X"
- "within this window"
- "after soaking"

👉 The moment you switch to that mindset, your system becomes much more powerful.

If you want next step, I can:

- upgrade your scheduler to a **true constraint engine (with priorities + deadlines)**
- or show how soak interacts with **time windows (morning/evening)**, which gets really

interesting.
29/03/2026, 03:09:22



How would I make it so it automatically schedule each day, I am running fast API for now so it can receive commands from the internet

29/03/2026, 10:36:02



Nice — now you're moving from "algorithm" to an actual **running system**.

You basically need three things:

1. **A daily planning trigger**
2. **A persistent state (zones, moisture, last watering, etc.)**
3. **An execution loop (runs valves at the right time)**

Let's build a clean, production-friendly approach for your FastAPI setup.

High-level architecture

```
Internet → FastAPI (API)
      ↓
    Scheduler (daily plan)
      ↓
  Stored Schedule (DB)
      ↓
    Executor (runs in real time)
```

👉 Separate:

- **Planning (once per day)**
- **Execution (real-time)**

1. When should scheduling run?

✅ **Best practice: once per day (rolling plan)**

Example:

- every day at **03:00**
- generate schedule for today (or next 24h)

Why:

- you can include fresh weather data
- you can adapt moisture daily



2. How to trigger it in FastAPI

FastAPI itself doesn't schedule tasks. You need a background scheduler.



Option A (simple & good): APScheduler

Use:

```
pip install apscheduler
```

Example integration

```
from fastapi import FastAPI
from apscheduler.schedulers.background import BackgroundScheduler

app = FastAPI()
scheduler = BackgroundScheduler()

def daily_planning():
    print("Generating schedule...")
    # call your planner here
    generate_daily_schedule()

scheduler.add_job(daily_planning, "cron", hour=3, minute=0)
scheduler.start()
```

👉 This runs automatically every day.



3. Store the generated schedule



You should NOT keep schedule only in memory.

Use:

- SQLite (simple)
- or PostgreSQL (better later)

Example structure

```
ScheduleEntry:
    start_time (datetime)
    zone_name (str)
    duration (int)
    executed (bool)
```

4. Execution loop (critical piece)

This is separate from planning.

It:

- checks current time
- starts zones when needed

Simple executor (runs every few seconds)

```
import time
from datetime import datetime

def executor_loop():
    while True:
        now = datetime.now()

        entries = get_pending_entries(now)

        for entry in entries:
            run_zone(entry.zone_name, entry.duration)
```

```
mark_executed(entry)

time.sleep(5)
```

Run it in background

```
import threading

threading.Thread(target=executor_loop, daemon=True).start()
```

5. Daily planning logic

Your `generate_daily_schedule()` should:

Step 1

Fetch:

- zones
- intents
- last moisture state

Step 2

Fetch environment:

- weather API
- temperature, rain

Step 3

Compute:

- ET
- adjusted durations

Step 4

Build schedule:

- your scheduler (cycle & soak)

Step 5

Save schedule to DB



6. Keep state across days

You MUST persist:

- soil moisture per zone
- last watering time

Otherwise your “smart” system resets every day ❌



Example flow over 2 days

Day 1 (03:00)

- generate schedule
- run watering
- update moisture

Day 2 (03:00)

- reuse updated moisture
- adapt schedule

👉 this is where intelligence appears



7. Add API endpoints

With FastAPI you can expose:

Force re-plan



```
@app.post("/plan")
def trigger_plan():
    generate_daily_schedule()
```

View schedule

```
@app.get("/schedule")
def get_schedule():
    return load_schedule()
```

Manual override

```
@app.post("/run-zone")
def run_zone_now(zone: str, duration: int):
    run_zone(zone, duration)
```

⚠ Important pitfalls

❌ Don't generate schedule "on the fly"

→ too late, unreliable

❌ Don't block FastAPI thread

→ always use background jobs

❌ Don't rely only on RAM

→ restart = lost schedule

🚀 Optional upgrade (very worth it)

Rolling horizon planning

Instead of “today only”:

- always maintain next **24–48h schedule**

Every few hours:

- recompute
- adjust

👉 this makes system resilient

🔥 Clean mental model

You now have 3 loops:

1. Planning loop (daily)

decides what should happen

2. Execution loop (seconds)

makes it happen

3. Adaptation loop (after watering)

updates soil model

🏁 Final answer

To automatically schedule each day:

👉 Add a **background scheduler (APScheduler)** to your FastAPI app that:

1. runs once per day
 2. generates a full schedule
 3. stores it in DB
 4. a separate loop executes it in real time
-

If you want next step, I can:

- show a **complete working FastAPI app (with DB + scheduler + executor)**

- or help you design this to run on a **Raspberry Pi controlling valves** (very fun part)